

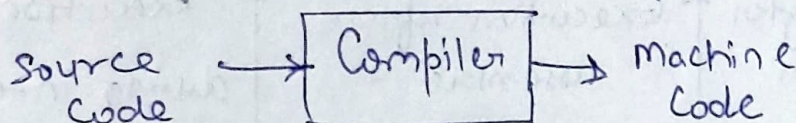
* Unit -1 Introduction *

Compiler :- A compiler is a program that translates a source program written in one language into an equivalent program in another language, called target language.

Basically, compiler is used to convert High level programming language into low level (machine) language.

Source language $\Rightarrow$ C, C++, java, python etc.

Target language $\Rightarrow$ Machine code, Assembly code etc.

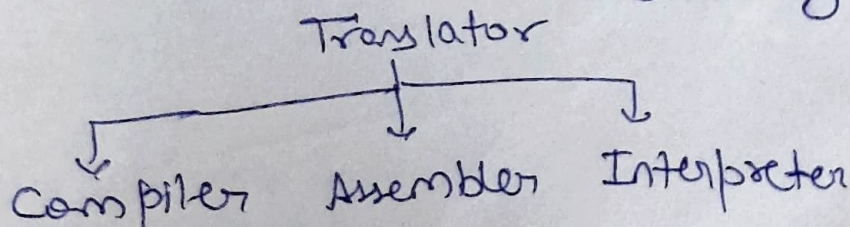


Main Work

- checks whether a program is correctly written
- detects error
- Analyzes program structure, check meanings.
- Create intermediate code & Optimized code.
- generate target code.

$\rightarrow$ Compiler acts as a bridge between programmer & computer hardware.

* Translator $\Rightarrow$ Translates source language into an equivalent target language.



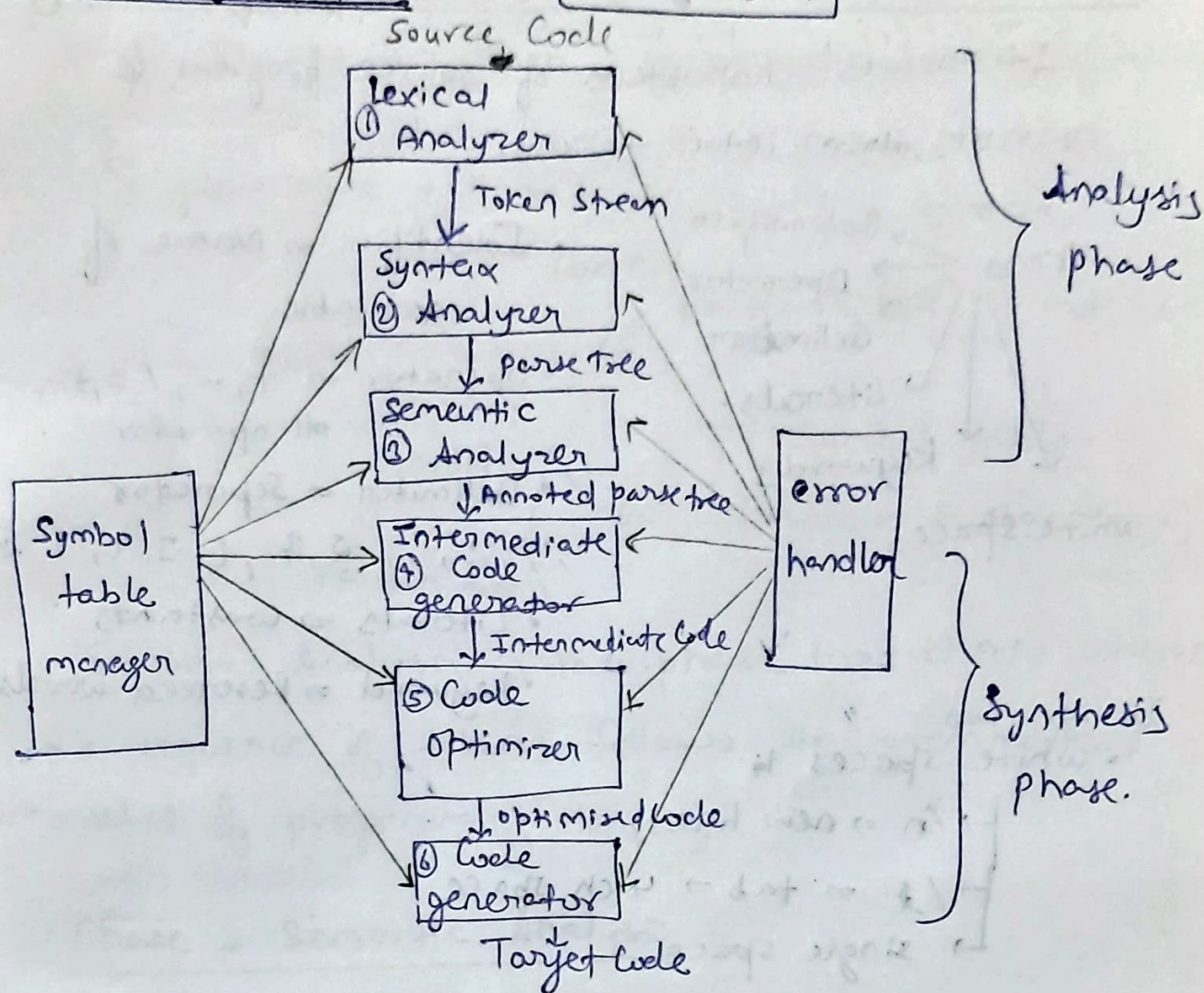
* Overview of Phases of compiler :-

* Difference between Compiler, Assembler & Interpreter

Compiler	Assembler	Interpreter
High level lang. ↓ machine code	Assembly lang. ↓ machine code	High-level lang ↓ execution
① whole source program is translated at once	Translates assembly instruction.	One statement at a time duration execution.
② Execution after compilation	Execution after assemble	Execution of during interpretation.
③ Reports errors during compilation	Reports assembly errors	Report when error is encountered during interpretation & stop there.
④ Generally faster compilation	Fast, because machine code is produced.	Slower
Ex: C/C++ compiler	Ex: MASM, NASM	Ex: Python compiler interpreter

* Phases of Compiler

LESSION



- Analysis Phase To understand the structure & meaning of the source program by analyzing.

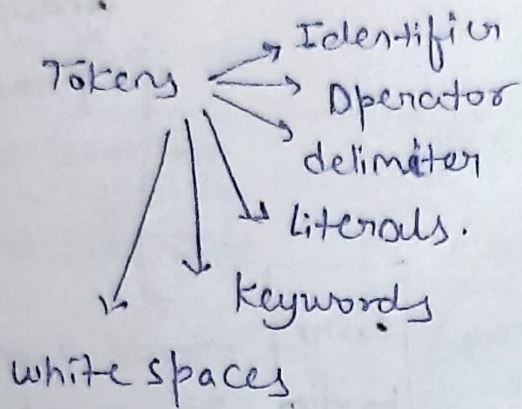
- ↳ Lexical Analyzer
- ↳ Syntax Analyzer
- ↳ Semantic Analyzer.

- Synthesis Phase Generate target program using Analyzed information.

- ↳ Intermediate Code generator
- ↳ Code optimizer
- ↳ Code generator

Phase 1 Lexical Analysis (Scanner) / (Linear Analysis)

It reads characters of source program & convert them into tokens.



- Identifier $\Rightarrow$ Name of variable.
- Operator $\Rightarrow +, -, /, =, +=,$ all operator
- Delimiter $\Rightarrow$ Separator
'', ''', '!', ' ', '\$', '%', '[', ']', '(', ')', etc.
- Literals $\Rightarrow$ Constants
- Keyword $\Rightarrow$ Reserved words

It ignores

- white spaces &

$\rightarrow$ $\backslash n$ $\Rightarrow$ new line
 $\rightarrow$ $\backslash t$ $\Rightarrow$ tab $\Rightarrow$ 4 ch space
 $\rightarrow$ single space

Sequence of character or group of character known as 'Lexeme'.

It categorized Lexeme with tokens $\Rightarrow$ Tokenization
 $\langle \text{Lexeme} : \text{tokens} \rangle$

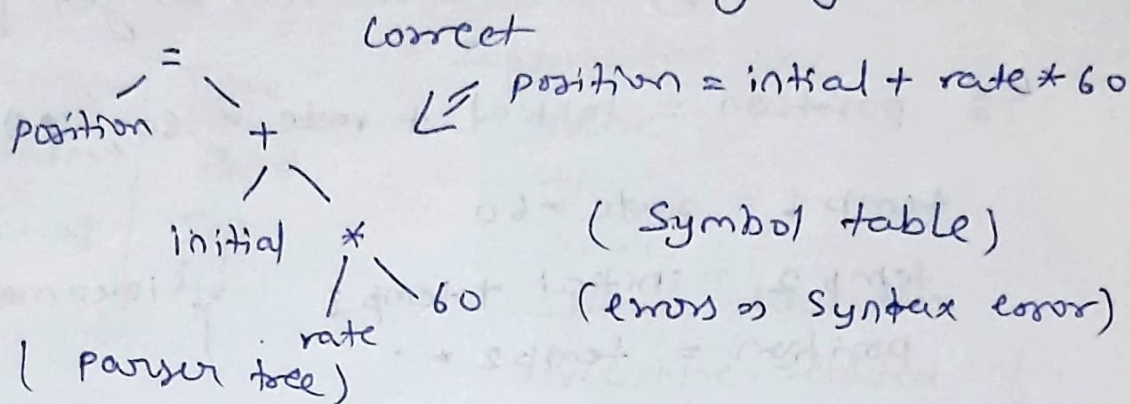
* output $\Rightarrow$ token Stream.

$\Rightarrow$ If error occurs then it throw "lexical error" to error handler.

Phase 2 Syntax Analysis (Parsing / Hierarchical)

checks grammar or grammatical order of tokens.

Ex: position = + * rate 60; wrong syntax.
Correct



Syntax Analysis is the phase that checks whether the sequence of tokens follows the grammatical rules of programming language.

Phase 3 Semantic Analysis

It checks - Type compatibility

- Meaning of operation
- Declaration of identifier
- Scope-related information
- Valid use of variable.

If error occurs then it is known as semantic error.

Ex: int rate;
rate = "hello";



As result
Annotated Parse
Tree

int rate;
rate = 60;



Phase 4 Intermediate Code generation

properties of Intermediate Code

- ↳ easy to produce
- ↳ Easy to translate into target program.

Ex: position = initial + rate * 60; // Original Code

temp1 = rate * 60

temp2 = initial + temp1

position = temp2

} Intermediate Code

Phase 5 Code Optimization

It makes code faster, smaller & more efficient without changing the result.

→ removes unnecessary operations from source code make it simple.

→ It makes resulting machine code execution faster.

Phase 6 Code Generation

memory locations are selected for each of the variables used by the program.

→ Each intermediate instructions are translated into a sequence of machine instructions the perform some tasks.

Ex: temp1 = rate * 60

temp2 = initial + temp1

⇒ move rate, R2
MUL #60.0, R2
etc

• Supporting Component

* Symbol Table :- User defined data structure which stores information about tokens of programs.

token ↓ Stream	Identifiers	Type	Information like
	position	float	
	initial	Int	
	rate	float	
			name type scope storage Allocation Procedure-related info.

Symbol table Allows us to find the record for each identifier quickly & to store/retrieve data from record.

* Error handler :-

Error like Lexical error, Syntax error & Semantic error in different phases are reported by error handler.

Syntax & Semantic phase handles large function of error.

Lexical	<token : error>
Syntax	"
Semantic	"

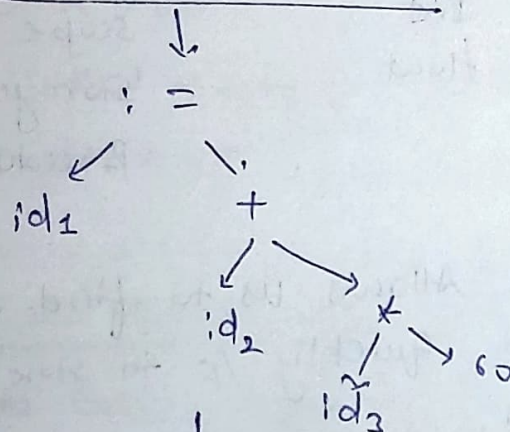
Ex

position := initial + rate * 60 (Source Code)

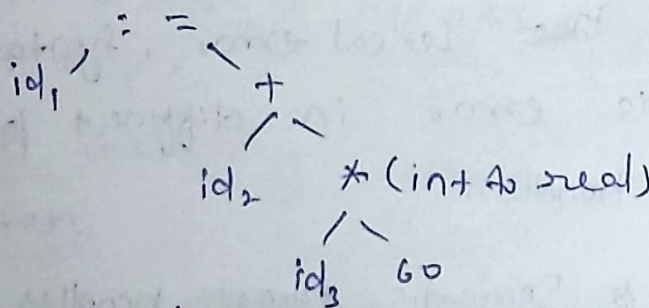
↓
Lexical Analyzer

↓
 $id_1 := id_2 + id_3 * 60$

↓
Syntax Analyzer



↓
Semantic Analyzer



↓
Intermediate Code generator

↓
temp1 := int to real(60)
temp2 := id₃ * temp1
temp3 := id₂ + temp2
id₁ = temp3

↓
Code optimizer

$\downarrow$
 $temp\ 1 := id_3 * 60.0$
 $id_3 := id_2 + temp_1$

$\downarrow$
 Code generator

$\downarrow$
 $mov\ id_3, R_2$
 $MUL\ \#60.0, R_2$
 $MOV\ id_2, R_1$
 $ADD\ R_2, R_1$
 $MOV\ R_1, id_1$

* Boot Strapping :-

Bootstrapping is the technique of producing a self-compiling compiler, i.e., a compiler written in a source programming language that it is intended to compile.

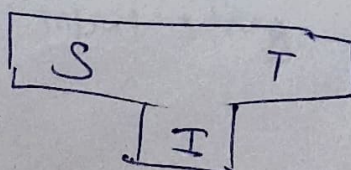
Note:- initially a compiler is built with the help of another existing compiler in simple language. then it compile source code for self.

Bootstrapping terms -

S $\rightarrow$ source language

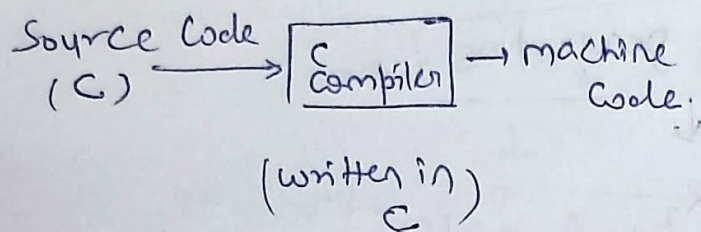
T $\rightarrow$ Target language

I $\rightarrow$ In which language compiler is written



Compiler Types

1. Self-compiling Compiler



Means Source code & implementation of compiler language is same.

* It is used in Bootstrapping.

Two more types - single pass, multiple pass, Compiler.

Advantage of Bootstrapping

1. Self-hosting compiler -

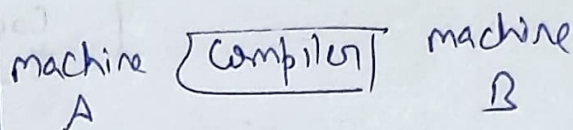
means compiler can be build & maintain in a language.

2. Portability $\rightarrow$ Can be used to create compilers for another machine by modifying the backend.

3. Compiler development

$\rightarrow$ useful in development in new programming language gradually.

2. Cross Compiler



means one machine run compiler to make code for another machine.

* Review of Finite Automata

Finite Automata is a mathematical model used to recognize whether a given input string belongs to a particular language or not. (Abstract Machine)

→ reads one input symbol at a time
then decides Accepted or rejected, validating with given patterns or rules.

$\underline{E_1}$ All strings ending with 0

110 ✓

100 ✓

111 ✗

Basic Terminology

(a) Alphabet - Σ finite set of symbols.

$\underline{E_1}$ $\Sigma = \{0, 1\}$ or $\Sigma = \{a, b\}$

(b) String $\Rightarrow$ finite sequence of Alphabet

for $\Sigma = \{0, 1\}$ $\underline{E_2}$ 0, 01, 1, 001, 0011

empty string $\in$ (length 0)

(c) Language $\Rightarrow$ set of strings over an alphabet (valid)

$\underline{E_3}$ $\Sigma = \{0, 1\}$

$L = \{0, 10, 110, 1110, 1010, \dots\}$

(d) Automation $\Rightarrow$ Machine / Model which process input string.

Finite Automata has generally 5 tuples -

$$M = (Q, \Sigma, \delta, q_0, F)$$

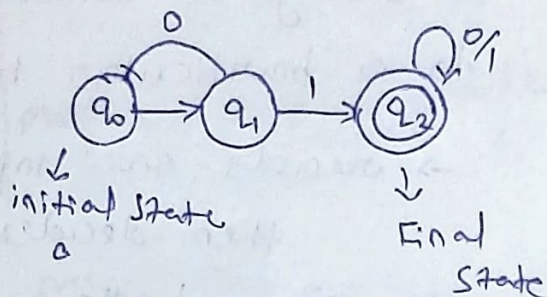
Q = Finite set of state,

Σ = set of input symbol

δ = Transition function

q_0 = Initial state

F = Final state



$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

Transition

Given current state + input symbol $\rightarrow$ Next state,

$$\delta(q_0, 1) \rightarrow q_1$$

Finite Automata have two types

DFA

(Deterministic Finite Automata)

NFA

(Non-Deterministic Finite Automata)

One transition for every state-input symbol pair

zero, one or multiple transition for every state-input symbol,

FA are used in Lexical Analysis to recognize token patterns to help the scanner identify tokens.

Lexical Analyzer :-

It is the first phase of a compiler that reads the source program & converts the sequence of characters into a sequence of tokens.

→ Grouping of character is done by scanner (Lex)

Primary work $ch \rightarrow group \rightarrow token$

Tokens

- keyword
- Identifier
- operator
- delimiter
- literal
- white space.

$i, n, + \Rightarrow$ character

int

$\hookrightarrow$ lexem.

Actual sequence/group of tokens, character

$int \Rightarrow$ keyword.

Category of lexeme

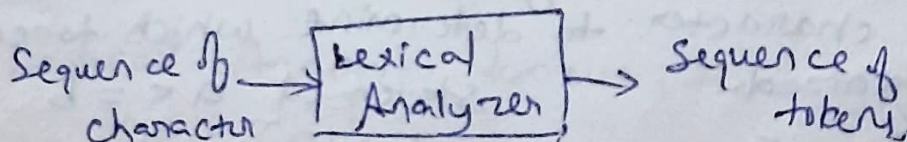
Pattern

Set of rule that define how a lexeme should be for token.

$\Rightarrow$ valid mobile number
 $(0-9)^+103$

$\left\{ \begin{array}{l} + \rightarrow \text{at least 1 time} \\ * \rightarrow 0 \text{ or more than 0} \end{array} \right.$

Token = Pattern + Lexeme

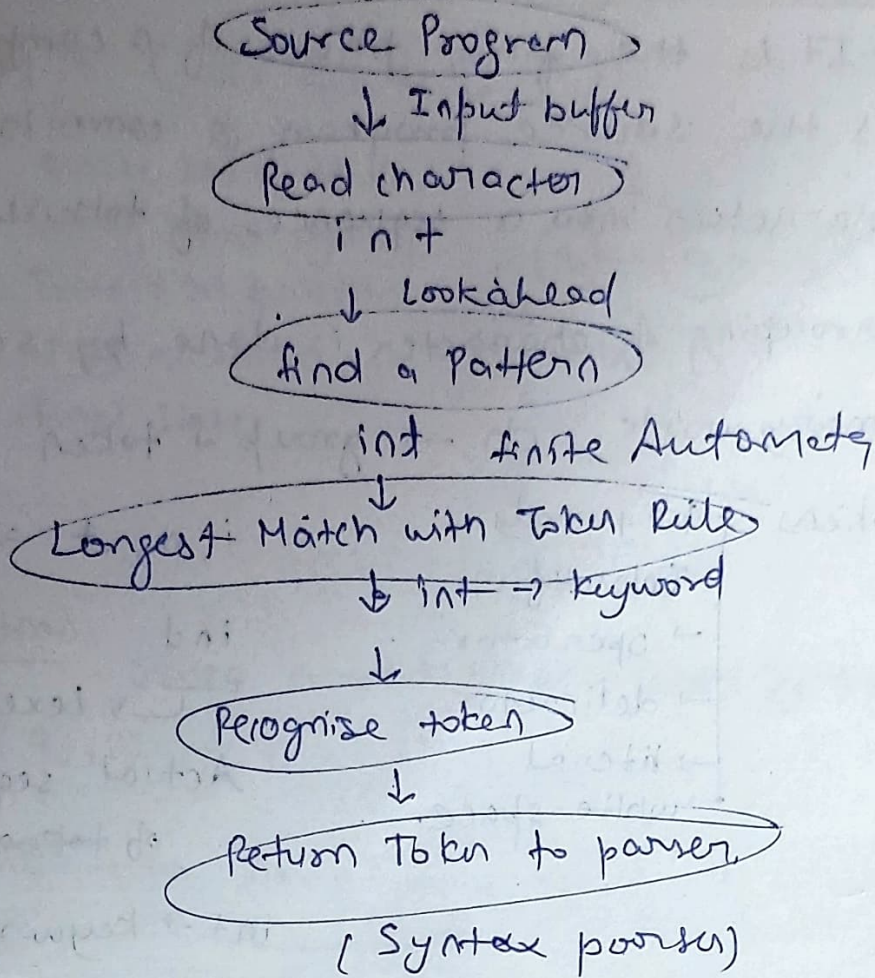


$a = b + 10 ;$

$\langle id \rangle \langle Assign \rangle \langle id \rangle \langle PLUS \rangle$

$\langle NUM \rangle \langle SEMICOLON \rangle$

Token recognition.



Pattern

Identifier $\Rightarrow$ Letter (Letter | digit)*

$L(LUD)^*$

Integer $\Rightarrow$ (digit)+

Scanner read until a white space occur then
or a terminator occur then it group then

Lookahead

means reading one or more upcoming
input character to determine which token is being
recognized.

$\leq$
 $<$
 $<=$

Statement $a \leq b$

when it reads $<$

it waits for next

$a < b$ $< ?$

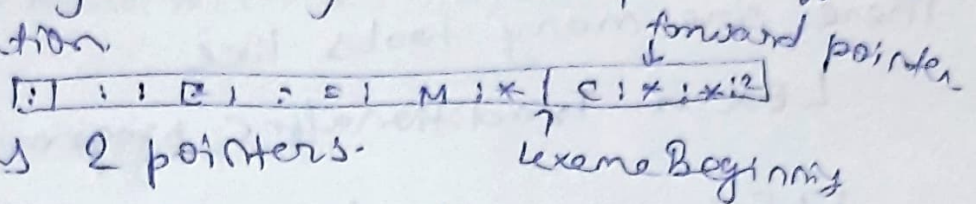
$a \leq b$ $< = ?$

Input Buffering

If lexical analyzer reads each & every character from file/disk then process will be slower.

So all the characters of source code are loaded into buffer.

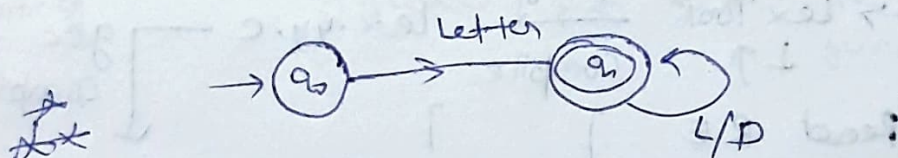
Then Lexical Analyzer use input buffer for tokenization.



→ It has 2 pointers.

1. LexemeBegin() → 1st ch of current lexeme.
2. forward → move input forward to identify token

Ex = int count = 25 ; → int → int
 ↑ ↑ ↑
 Lexeme begin forward
 at space, lexeme complete.



→ Scanner choose longest valid lexeme.

(Longest match / Minimal munch principle)

EOF ⇒ End of the file

Lexical Analyzer knows, there is no more token after 'EOF'.

LEX Specification File:-

1. Pattern → what to recognize
2. Action → what to do.

Structure

% \$

// C header files

// variable declare / initialize

definitions % {

// macro definition

FIX_VALUE = [6-9][0-9]{9}

%%

rules

// Rule section

// Pattern (in Pair pattern & Action)

{ MOBILE } { printf("valid mobile number"); }

% %

Sub-routine

// Userdefined subroutine

int main() {

yylex(); → generate lexical analyzer function.

}

Language Processor Tools (Sequence of tools)

